

Vehicle & Pedestrian Tracking in Video Sequences

Progress report

Gr 07

M.R.A. Rahman (E/23/282) · A. Piraveen (E/23/273) · M.M.M.A. Nular (E/23/249) · V. Thanush (E/23/392)

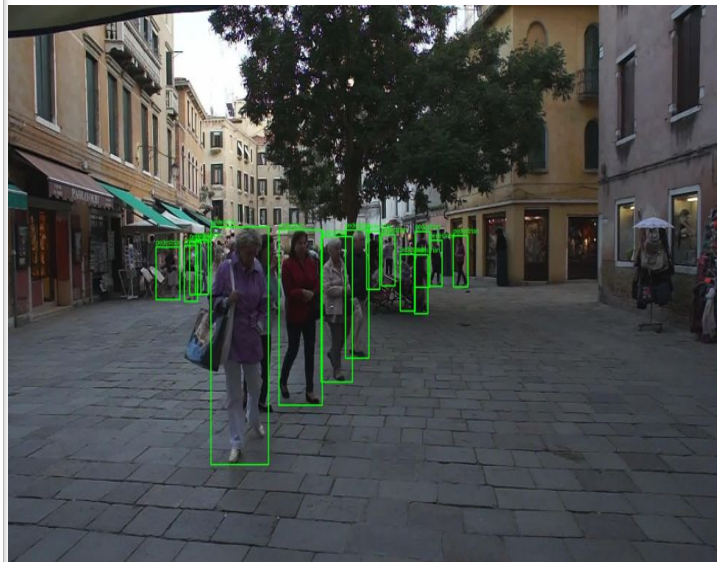
Dataset Status & Source

Dataset: MOT16 (from Kaggle)

Sequence	Frames	Resolution	FPS
MOT16-02	600	1920×1080	30
MOT16-04	1050	1920×1080	30
MOT16-05	837	640×480	14
MOT16-09	525	1920×1080	30
MOT16-10	654	1920×1080	30
MOT16-11	900	1920×1080	30
MOT16-13	750	1920×1080	25

7 test sequences (no GT — reserved for evaluation)

- Pedestrian bboxes (train + val): 79 799
- Total annotated frames: 5 316
- Visibility threshold applied: $\geq 30\%$
- Classes retained: Pedestrian (cls 1) · Car (cls 3)
- Source: Kaggle → data/raw/MOT16/



Preprocessing & Annotation Pipeline

- We developed a custom Python script (`preprocess.py`) to convert raw MOT16 tracking data into standard YOLO format.
- The pipeline reads sequence metadata (`seqinfo.ini`), extracts resolution and frame rate details, and filters out low-confidence annotations.
- Bounding boxes were converted from standard pixel dimensions (Left, Top, Width, Height) to normalized YOLO center coordinates (`cx`, `cy`, `w`, `h`) scaled from 0.0 to 1.0.
- To prevent temporal data leakage, we split each video sequence chronologically, assigning the first 80% of frames to training (4,254 frames) and the final 20% to validation (1,062 frames).
- We generated the required `dataset.yaml` configuration file and implemented a verification script (`verify_labels.py`) to visually confirm box accuracy and check boundary constraints.

Baseline Pipeline & Methodology

Core Methodology: Our system decouples object detection from tracking, feeding frame-by-frame detections into a multi-object association algorithm.

Two Stage Detection and Tracking Architecture

1. **Detection (YOLOv8):** We use a YOLOv8n detector fine-tuned on our MOT16 training split to localize pedestrians and vehicles in each 1920x1080 frame.
2. **Tracking Evolution:**
 - **Baseline (SORT):** Our initial tracker uses Simple Online and Realtime Tracking, which relies on Kalman filters for motion prediction and Hungarian matching for bounding box association without visual re-identification
 - **Advanced Upgrade (DeepSORT):** We are integrating DeepSORT to add deep feature embeddings, allowing the system to re-identify pedestrians after occlusions.
 - **High-Performance Upgrade (ByteTrack):** We will also evaluate ByteTrack, which utilizes a two-stage association process to recover low-confidence detections and reduce identity switches.

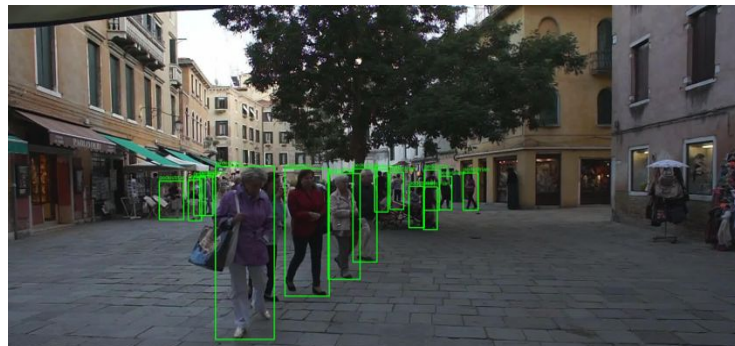
Initial Results & Current Challenges

Initial Progress (Qualitative):

- Successfully generated and verified 79,799 label files across 5,316 total frames.
- Visual verification confirmed accurate bounding box alignment in high-density scenes like MOT16-02.

Current Technical Challenges:

- **Severe Class Imbalance:** The MOT16 dataset is heavily pedestrian-dominated
- **Crowd Occlusion:** High pedestrian density creates overlapping bounding boxes,
- **Hardware Constraints:** Full YOLOv8 model training requires dedicated GPU acceleration,



Roadmap & Next steps

- **Milestone M3 (Target: July 28):** Install the Ultralytics framework, execute YOLOv8 training using our generated `dataset.yaml`, and produce baseline SORT tracking videos.
- **Milestone M4 (Target: August 15):** Integrate DeepSORT and ByteTrack architectures, and fine-tune confidence and IoU thresholds for each tracker.
- **Milestone M5 (Target: August 18):** Run quantitative evaluations using `py-motmetrics` to generate comparative MOTA (Multi-Object Tracking Accuracy) and IDF1 performance tables.
- **Milestone M6 (Target: August 25):** Conclude experiments, document failure modes and edge cases, and finalize project reporting.
- **Repository Status:** Our codebase and preprocessing scripts are fully up to date on [GitHub](#)